

Spring Cloud

Spring Boot is a convenient framework for Java programmers to develop microservices and cloud native apps on-the-fly. It can also create web applications with the embedded Tomcat or Jetty servers, thus reducing the deployment of WAR or EAR files. However, when designing Spring Boot applications in the cloud, there is additional overhead in terms of configuration and security setup. As more microservices are deployed in the cloud, the inter-service communication of these Spring Boot applications requires additional steps of service discovery, inter-service communication, and gateway security.

This challenge can be easily addressed by using Spring Cloud framework on top of Spring Boot model, providing inter-service communication and making it suitable

for cloud architecture. Configuration, circuit breakers, service discovery, API routing, messaging, monitoring and logging, tracing calls and service testing are integral features of Spring Cloud. These components are loosely coupled, reducing operational complexity. The circuit breaker pattern, derived from design patterns, ensures high availability, providing fault tolerance and resilience even when services experience downtime or slowdowns causing latency in calls.

Technically, Spring Cloud can be compared with service mesh like Istio as both address microservices architecture for inter-service communication, circuit breakers, and service discovery integrated framework. Spring Cloud has some advantages, such as annotation-based coding, flexible policy definition, and the ability to be stateful or stateless, but it adds overhead by adding tracing

data to logs by instrumenting service calls through network proxies.

API gateway vs service mesh architecture

When developing microservices on cloud platforms, architects draw an analogy between service mesh architecture and API gateway architecture. In reality, they serve different purposes and should not be compared in architectural design.

An API gateway is commonly used in API development, acting as the routing interface between API services and the client calls (requests). It can be utilised for service discovery, security features like IAM authentication, role-based access, response caching for faster request processing, and integrated logging and monitoring services.

In contrast, a service mesh is a network of proxies for services, also called sidecars, improving communication between service-to-service calls. In network terminology, client-to-server calls are called north-south traffic and inter-communication of servers is called east-west traffic. In other words, API gateway handles north-south traffic and service mesh handles east-west traffic.

Service mesh handles the internal brokering of resources, sitting between network communication channels and application services to accelerate

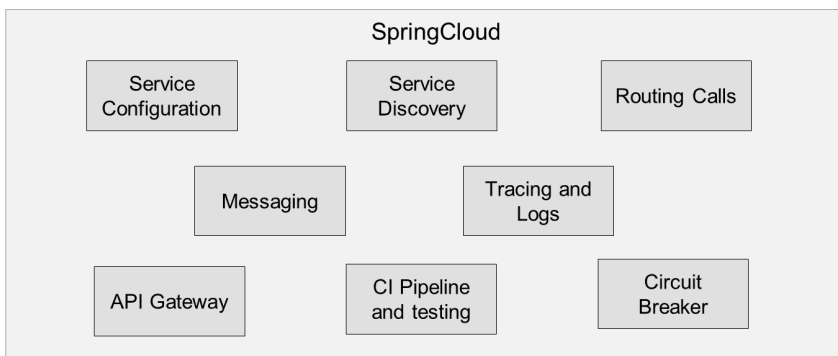


Figure 2: Spring Cloud

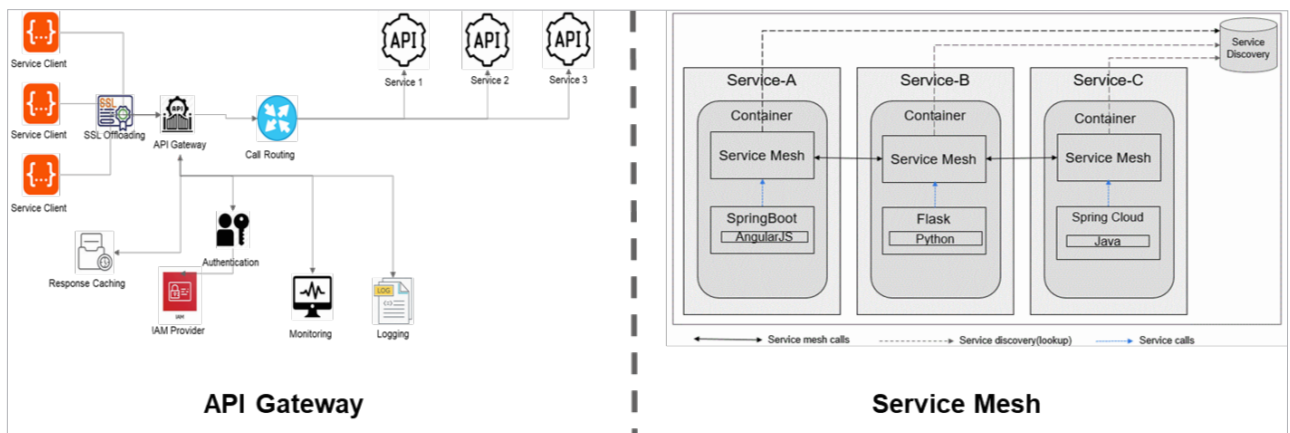


Figure 3: API gateway vs service mesh